

Sistemas multiagente: Comunicación basada en gossiping

Miguel García Bohigues

December 2022

1 Introducción

La comunicación en los sistemas multi-agente es algo crucial para su correcto funcionamiento. La naturaleza colaborativa de los agentes hace que sea necesario que lleguen a acuerdos unos con otros para poder llevar a cabo acciones en grupo y no como unidades aisladas. Existen múltiples protocolos de comunicación entre agentes para conseguir llegar a estos consensos, no obstante no todos son aplicables a todos los escenarios y debemos buscar aquellos que se ajusten mejor al problema que estamos tratando.

En la presente memoria revisaremos tres posibles algoritmos de comunicación entre agentes para llegar a un consenso sobre un valor. El valor en este caso es incontextual, es decir, no hace referencia a un caso real. No obstante se puede ver su aplicación fácilmente siendo que, si conseguimos que los agentes se pongan de acuerdo sobre un valor arbitrario, pueden acordar entre todos la temperatura de una habitación, el precio medio de un producto, etc.

En concreto, vamos a trabajar el denominado *gossiping* o chismorreo en castellano. Se trata de un protocolo de comunicación en el que un agente se comunica con un grupo reducido del conjunto total de agentes que se encuentran en el sistema. Entre ellos llegan a un acuerdo y proceden a hablar con otro subgrupo dentro de la red. Los agentes siguen este proceso hasta que todos comparten el mismo valor.

2 Gossiping

Como hemos comentado en la introducción, el mundo de la comunicación entre agentes es muy rico, disponemos de muchas posibles implementaciones para un mismo tipo de comunicación. En este trabajo vamos a revisar tres aproximaciones: envío (*push*), demanda (*pull*) y conjunta (*push-pull*)

A lo largo de todas las implementaciones se ha utilizado la misma herramienta. Los sistemas se han desarrollado sobre el framework SPADE, que sirve como capa de abstracción para poder realizar plataformas multi-agente.

SPADE está programado en Python, por lo que este ha sido el language de programación escogido para el desarrollo.

Cada uno de los modelos de agente que comentaremos a continuación se ha replicado para diferentes números de agentes dentro de la red. En concreto se han probado redes de agentes de 10, 20, 100 y 500 agentes. Para probar la carga del sistema y la convergencia a un consenso de los diferentes modelos y modos de actualización

3 Implementación por envío

El *gossiping* por envío tiene como premisa la comunicación unidireccional. Un agente comunica a otro/s su creencia sobre el valor. En este escenario se envía un mensaje por interacción y no existe respuesta alguna. Como hemos comentado previamente, este proceso es periódico hasta que se llega a un consenso.

En este sentido se ha desarrollado un modelo de agente que, periódicamente envía un mensaje con su creencia actual sobre el valor. En caso de recepción de un mensaje, comprueba ambos valores y actualiza su creencia en función del modo escogido. Existen dos modos de actualización configurables: máximo o punto medio. Respectivamente, el agente se quedará con el número más alto o con la media de ambos valores.

A continuación se ofrecen unas gráficas comaprativas del número de iteraciones que han sido necesarias para conseguir convergencia para ambos casos. Se muestran además para diferentes tallas de la red de agentes.

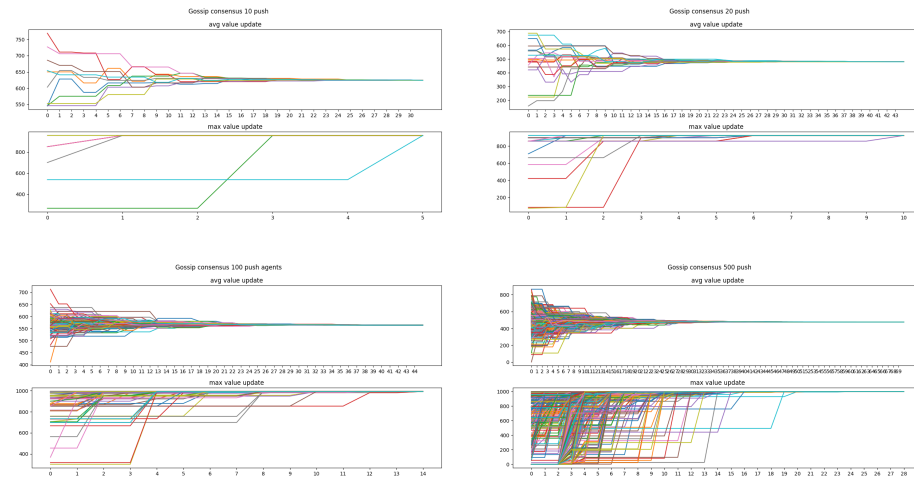


Figure 1: Evolución del valor para agentes push

Como podemos ver en la figura anterior, a medida que aumenta el número de agentes de la red, el consenso cuesta más de conseguir. Para un caso trivial como el de 10 agentes, la conversión se consigue en 30 intercambios actualizando

según el valor medio y en cinco si actualizamos según el valor máximo. Por otro lado, vemos como para la talla más elevada la convergencia se obtiene tras 69 interacciones para la media y tras 30 para el máximo. Este resultado es coherente con la realidad, puesto que es más fácil llegar a un acuerdo si el grupo es reducido.

Estas pruebas las hemos realizado asumiendo que cada agente se comunica únicamente con un solo vecino en cada momento. Pero esto no tiene por qué ser así. Para redes grandes, podemos dividir las en subgrupos de agentes para llegar a consensos locales que luego transmitir al resto de la red. Para desarrollar esta prueba hemos parametrizado el número de vecinos de un agente y hemos realizado pruebas. Para estas hemos considerado que el tamaño de la red más interesante debería ser 100, puesto que sus gráficas presentan claridad a la vez que complejidad.

En este sentido hemos probado con 1, 5, 20 y 50 vecinos. El primer caso es el que podemos ver en la figura anterior. Y en la figura siguiente se puede observar lo obtenido con el resto de valores.

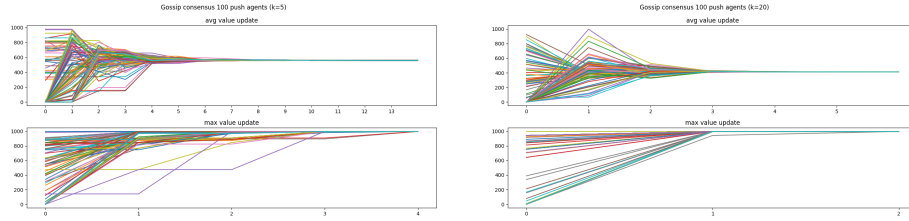


Figure 2: Convergencia según el número de vecinos

Como vemos, con 5 vecinos hemos conseguido reducir los 44 pasos necesarios inicialmente a 13. Además con 20 vecinos lo hemos reducido a tan solo 6. Esto nos hace pensar que, cuantos más vecinos mejor, ¿no?. Pues en efecto, no. Para 50 vecinos la solución, tras más de 100 iteraciones no ha conseguido converger. La red ha resultado en una gran mayoría de los agentes con un consenso, pero con unos pocos disidentes. Se llega muy rápido a un valor cercano al máximo o medio (según el caso), pero no se llega a conseguir el consenso.

4 Implementación por demanda

En este caso, el *gossiping* toma un enfoque bidireccional parcial. En esta aproximación los agentes en lugar de enviar su valor sin consultar, solicitan a un agente vecino que le envíe su valor para así actualizar el propio si fuese necesario. Aquí pasamos de un mensaje por interacción a dos.

El comportamiento comentado previamente se ha implementado en *SPADE* y se han obtenido los resultados para las mismas tallas de red. No obstante, para ofrecer comparativas entre implementaciones vamos a centrarnos solo en la red de tamaño 100. Tal y como hemos justificado previamente, es la adecuada por

igualdad de complejidad y claridad en los resultados. El método por demanda ha conseguido la siguiente convergencia:

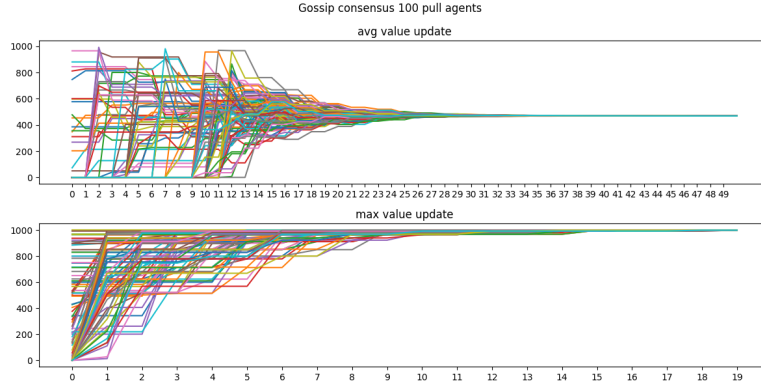


Figure 3: Evolución del valor para agentes pull

Como podemos ver, los resultados de convergencia son peores que en el caso unidireccional. Esto es coherente puesto que al no ser una comunicación directa, sino que presenta un esquema de solicitud-respuesta, el tiempo que ha de pasar para obtener el consenso es mayor. Esto lo vemos también reflejado en el caos de utilizar más vecinos. En la siguiente figura se muestran los resultados para 5 y 20 vecinos. Para 50 se ha seguido sin conseguir convergencia.

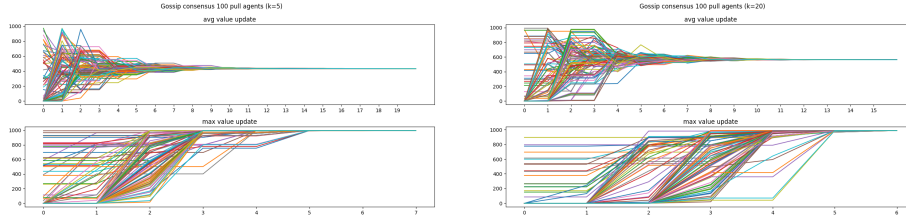


Figure 4: Convergencia según el número de vecinos

Como vemos la diferencia de períodos hasta llegar a convergencia sigue viéndose patente a pesar de incrementar el número de vecinos.

5 Implementación conjunta

En esta última implementación, hemos considerado un *gossiping* plenamente bidireccional. En este caso los agentes tienen dos protocolos de comunicación. Uno que periódicamente envía mensajes a sus vecinos, y otro que periódicamente solicita a sus vecinos su valor. Bien sea por haberlo solicitado o no, el agente al recibir un nuevo valor actualiza el suyo en función del modo que hayamos escogido (valor máximo o medio).

En este caso, de media para realizar una actualización de valor se dan menos de 2 mensajes. Puesto que el componente push del agente hace que se pueda recibir un valor sin haberlo solicitado. Acelerando la versión por demanda.

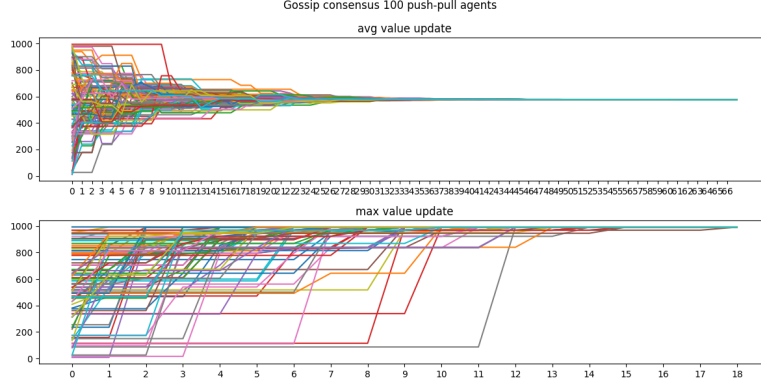


Figure 5: Caption

Para complementar la comparación, hemos reproducido los experimentos con los números de vecinos que hemos ido comentando durante la memoria, 5 y 20. Los resultados se muestran en la figura siguiente. En ellas podemos observar cómo la aproximación con 20 vecinos mejora a las dos variantes expuestas previamente.

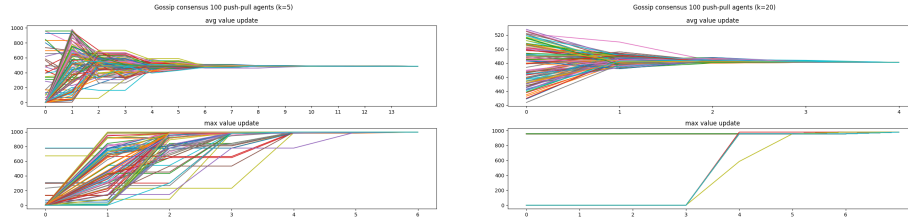


Figure 6: Convergencia según el número de vecinos

6 Conclusiones

En esta memoria hemos visto una revisión de tres implementaciones diferentes de comunicación basada en *gossiping*. En cada una de ellas hemos visto su funcionamiento y ventajas respecto a las demás.

Hemos visto cómo la comunicación por envío ofrecía mejores resultados que la comunicación por demanda. Siendo que esta primera requiere menor número de comunicaciones y, como resultado, la actualización del valor se lleva a cabo en menos tiempo.

No obstante, hemos observado también como la mejor aproximación que hemos obtenido no ha sido ni envío ni demanda, sino la combinación de las dos. Esta ha reflejado una convergencia realmente rápida, siendo que combina lo mejor de ambas aproximaciones. Presenta la rápida propagación de la política de envío junto a la solicitud de valores. Esto hace que se pueda realizar una actualización del valor mientras hacemos una solicitud, fruto de que un vecino nos haya comunicado un valor mejor.

Siendo que el desarrollo de las tres soluciones es muy similar en cuanto al coste, concluimos que en función del rendimiento y la consistencia de resultados, la mejor aproximación para que los agentes lleguen a un acuerdo es la implementación combinada de los algoritmos de envío y demanda.